Pramaan: A Self-Grading Research Agent for Indian Company Filings That Learns From Exchange XBRL

Atharva Rege

October 4, 2026

Abstract

Indian listed companies file the signed results PDF with their board meeting outcome and, shortly afterwards, the same numbers as machine readable Integrated Filing XBRL. The PDF arrives first and the XBRL follows, which makes the XBRL a free, automatic answer key that refreshes every quarter and cannot appear in any model's training data. We build Pramaan, a research agent that answers analyst questions from the PDFs, cites the page and the printed figure behind every number, and verifies each answer before showing it. We contribute (i) a question gym whose gold answers, gold pages and graders are generated from XBRL, including the finding that exchange XBRL and the printed row are defined differently for some line items; (ii) a retrieval cascade whose cross-encoder is fine-tuned on free page labels; (iii) a planner-executor agent whose models never do arithmetic, gated by deterministic verification, with a small-to-strong cascade and an abstention analysis; and (iv) a paired evaluation that reports its negative results. On a dry run on one quarter, the best specified configuration, `strong_only`, reaches 70.0% verified accuracy against 9.0% for naive retrieval augmented generation on a fixed sample of 100 test questions. The preregistered cascade hypothesis (H1) is *not supported* on the test sample; on dev we traced the cause to two concrete defects, a verifier rule that discards correct percentage answers and a retry rule that bans the page holding the answer, and evaluate exploratory fixes (1 of 3 is confirmed on the held-out test sample under the preregistered rule: `cascade_keep_v3`). Learning from graded outcomes (prompt evolution, a validated lesson memory) and a prompt-injection evaluation are left to future work. Section 7 states what the numbers do and do not support.

1 Introduction

Question answering over financial filings is difficult for reasons that general document QA does not capture: numbers are printed in tables whose unit (crore, lakh, million) is stated once in a header, standalone and consolidated statements sit side by side or on adjacent pages, rows carry near identical labels, and many PDFs are scans with an embedded OCR layer that corrupts digits. A wrong digit or a missed unit is a confident, expensive error. We therefore treat *verifiability* as the primary property: an answer counts only if it is correct *and* its citations check out.

Since April 2025 a listed company files its results PDF and the matching Integrated Filing XBRL on the same date. We use the lag between them as a design principle: answer from the PDF when it lands, grade yourself when the XBRL arrives, and learn from the outcome. This paper describes the system, the benchmark it generates, and a first evaluation on one quarter. We state hypotheses before looking at test data, report paired statistics, and keep specified configurations separate from variants that we designed after reading dev failures.

2 Related work

Financial QA benchmarks such as FinanceBench [8] are fixed and public, so they can leak into training data; to our knowledge no automatically refreshing benchmark exists for Indian company filings. Retrieval pipelines that fuse lexical and dense results [3] and rerank with cross-encoders [10] are standard; we show that an off-the-shelf reranker can hurt on near identical statement pages and that free page labels fix it. Our agent modules are DSPy programs [9]; reflective prompt evolution [1] and verbal reinforcement [11] are natural ways to learn from the graded outcomes and are future work here. Cascades trade cost for accuracy [2]; ours is gated by deterministic checks rather than a learned scorer. Selective prediction and calibration [4, 6] frame our abstention analysis. Indirect prompt injection [5] matters for any agent that reads third-party documents; the agent marks page text as data in the manner of spotlighting [7], but we do not evaluate robustness here.

3 The gym

Gold from XBRL. Per filing we store the PDF, the standalone and consolidated XBRL instances, and both publication timestamps. Questions come from six tiers: lookup, year on year growth, ratio, cross period share, cross company ranking, and abstain or false premise (including controls where the premise is true, so that abstaining on everything does not score). Gold values are computed from XBRL; the grader tolerates half the printed precision for money and 0.1 points for percentages, and ranking questions must match the exact order.

Only ask what the PDF prints. We locate each fact in the PDF by rendering it under unit candidates and decimals and matching table cells, calibrating the page unit by the number of matches. A fact becomes a question only if its cell is in the row for that line item; if the page has an exact label match with a different number the fact is dropped and counted. This both guarantees valid gold and surfaces a real data issue: for some consolidated statements the XBRL profit before tax excludes the share of associates that the printed row includes.

As-of guard and splits. Every question has an as-of timestamp and tools only see documents published before it. Splits are chronological and frozen; the test split is guarded in code and every access is logged. Question selection fills the six tiers proportionally and caps questions per filing; when the cap binds, the tier mix is preserved rather than letting the abundant lookup tier crowd out the others.

4 System

Retrieval. Page documents (a digest of the table for results pages) and row documents are indexed by BM25 and a dense encoder; reciprocal rank fusion feeds a cross-encoder, optionally fine-tuned on pairs (question, printed page) with hard negatives from the same filing, the other scope, other periods and other companies, holding out companies for early stopping. A hard check refuses to export any page that belongs to a dev or test filing.

Agent. Intake, plan, execute, compose, verify, reflect, escalate, abstain, implemented as a plain Python state machine around DSPy modules. Tools are typed and respect the as-of guard; arithmetic happens only in an AST calculator. Table reading is a DSPy module over layout text or a

deterministic layout parser; normalisation (unit, period columns, canonical line item, conversion to crore) is deterministic. Page reads are shared across questions and single-flight, so concurrent workers never pay for the same page twice.

Verification. V1 quote on the cited page and equal to the input under the page unit; V2 recomputation; V3 unit sanity and magnitude against the company's previous filing; V4 accounting identities on the cited table; V5 period; V6 XBRL cross-check in production; V7 cell provenance. A hard failure triggers one reflection with the small model, then escalation to the strong model, then abstention. Confidence is a logistic model over check results, retrieval margin, tier and retries.

5 Failure diagnosis

One diagnostic function compares an episode with gold slots, gold pages and gold values and writes plain-language feedback (for example "value is 100x too large; the page unit is lakhs"). Failure attribution assigns each failure to a stage and replays the episode with the gold artifact injected at that stage; it has been run on the offline agent only. The same feedback is designed to drive prompt evolution and a lesson memory, which are future work.

6 Experiments

6.1 Setup

All results come from a dry run on the quarter ended 30 June 2026, because the September quarter had not been filed when this version was written. The gym covers 245 filings and holds 483 train, 291 dev and 295 test questions; splits are chronological by board meeting date and frozen. Unless stated otherwise, agent results use one fixed random sample of 100 of the 291 dev questions for every configuration, so differences are paired. Only a preregistered sample of 100 of 295 test questions has been read; the other test questions are unread. The guard log records 10 test reads: one per configuration, plus those of a first attempt that a budget-accounting bug invalidated and that was re-run with unchanged settings. The small and strong tiers are `claude-haiku-4-5` and `claude-sonnet-5-5`. *Verified accuracy* counts an answer only if it is correct and its citations check out. Confidence intervals are 95% bootstrap intervals clustered by filing; paired differences use the same clustered bootstrap and McNemar's test.

6.2 E1: is the gym valid?

When the rules reader found the printed row, the PDF and the XBRL agreed on 88.5% of 2593 facts. A further 1874 facts had no matching row and 606 no unit header, so they say nothing about agreement. 62 facts were dropped because the XBRL element and the printed row are defined differently (for example consolidated profit before tax with and without the share of associates). Gold evidence was located for 89.6% of 719 facts on digital pages, 77.9% of 4354 on pages with an embedded OCR layer, and almost never (2.9%) for the 38 filings where no results page was detected; those filings contribute no questions. Human validation of the 100-question sample has not been done.

6.3 E2: what does each retrieval stage buy?

Dense retrieval alone beats BM25 by a wide margin on numeric table pages, and fusing the two with reciprocal rank fusion is *worse* than dense alone on nDCG@10 (0.829 against 0.911). Row documents, which render each table row as a retrievable unit, lift nDCG@10 to 0.939. The off-the-shelf cross-encoder then *lowers* it to 0.745, because it cannot tell near identical statement pages apart. Fine-tuned on 2922 train-split pairs whose labels come from XBRL for free, it reaches 0.987 (recall@1 0.488 to 0.976). The fine-tuning pairs and their negatives come from train filings only; an earlier version drew negatives from dev and test pages and was discarded and retrained, which changed nDCG@10 by less than 0.002. Queries are synthetic template questions from one quarter, so this measures generalisation to new companies and filings, not to free-form analyst wording.

stages (company-filtered)	R@1	R@5	nDCG@10
R1 BM25 only	0.440	0.860	0.694
R1 dense only	0.797	0.990	0.911
R1+R2 BM25+dense RRF	0.633	0.986	0.829
R1+R2 with row docs	0.855	1.000	0.939
R1-R3 + off-the-shelf cross-encoder	0.488	0.918	0.745
R1-R3 + fine-tuned cross-encoder	0.976	0.990	0.987

Table 1: Retrieval stages on 207 dev queries.

6.4 E3: the agent against baselines

config	n	verified acc. (CI)	abstain	$/question	$/verified correct	p50 s
`B0_naive_rag`	100	10.0 (5–16)	87	0.0025	0.0253	0.1
`small_only`	100	71.0 (61–80)	33	0.0146	0.0205	19.1
`strong_only`	100	80.0 (68–90)	26	0.0305	0.0381	14.9
`cascade`	100	71.0 (61–80)	33	0.0180	0.0254	15.3

Table 2: Specified configurations on the same dev sample. Costs are real API spend recorded at the original price of cached calls; latency is wall clock plus the recorded latency of cached calls.

The agent beats the naive RAG baseline by +61.0 points (95% CI +50.0 to +70.1; 62 fixed, 1 broken; McNemar $p < 0.001$). Naive RAG is cheap per question but almost always wrong or silent, so per verified correct answer the small-model agent is cheaper ($0.0205 against $0.0253). The long-context baseline B1 was not run: it sends whole PDFs, and the cost estimator puts it far above the budget of this study, so the second half of H3 is untested. The strong model gains +9.0 points (95% CI -2.9 to +20.5; 16 fixed, 7 broken; McNemar $p = 0.093$) over the small one, a difference that does not reach significance at this sample size. **H1 is not supported as preregistered.** The specified cascade differs from `strong_only` by -9.0 points (95% CI -20.5 to +2.9; 7 fixed, 16 broken; McNemar $p = 0.093$), so the preregistered condition (within two points of `strong_only` and cheaper) fails on accuracy, and it is dominated by `small_only` (it gave identical verified answers on every question, at a higher cost per question). Reflection ran on the questions that hard-failed verification but ended in "value not found" abstentions, and escalation fires only on a hard failure after reflection, so the strong model was consulted on almost no questions.

Offline baseline. A deterministic rules reader with no language model (clearly labelled offline, on all 291 dev questions) reaches 50.5% with the off-the-shelf reranker and 60.1% with the fine-tuned one. It is precise but silent: it abstains on about half of the answerable questions.

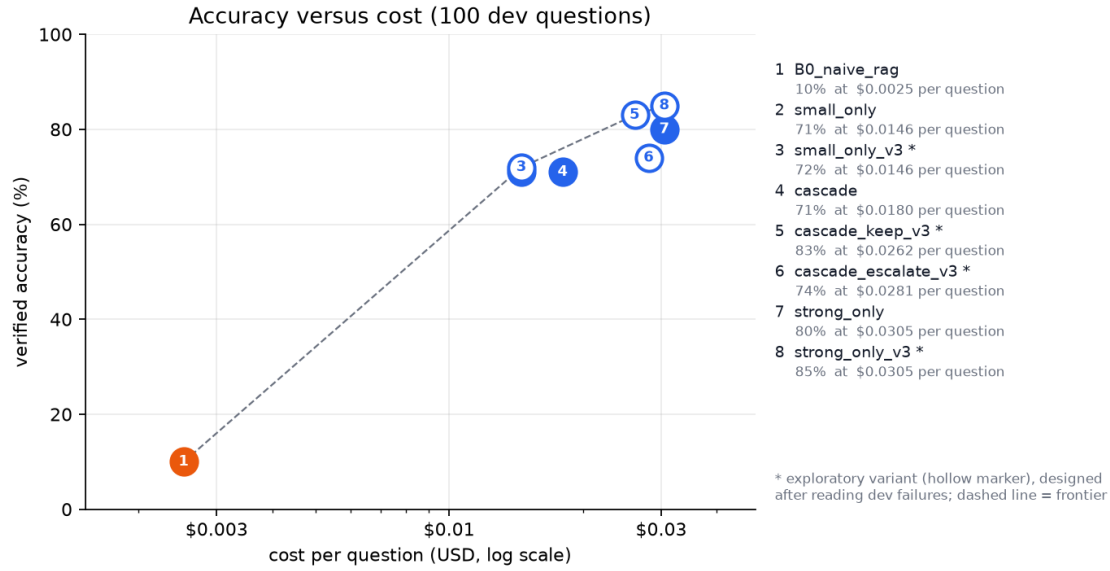


Figure 1: Verified accuracy against cost per question. Hollow markers are exploratory variants.

6.5 Exploratory: why the cascade did not help

*The variants in this subsection were designed after reading failures on the dev sample, so their numbers are optimistic and are marked *; they need confirming on questions not used to design them, and the specified configurations stay in every table.*

The verifier discards correct answers. Grading the stored drafts of abstained answerable questions shows that `strong_only` threw away 6 correct answers (V3 5, V4 1) and `small_only` 4 (V2 2, V3 1, V4 1); 12 and 4 further questions ended with no numeric draft at all because the model itself reported the value as not found. All 6 V3 (unit sanity) false rejects have one shape: a percent answer computed from two inputs cited on a single page. There the page's unit header cancels, so a missing header cannot hide a scale error. Relaxing V3 for exactly that case (inputs on different pages and absolute values keep the strict rule) is the variant `small_only_v3` / `strong_only_v3`. It recovers +5.0 points (95% CI +0.0 to +11.2; 5 fixed, 0 broken; McNemar $p = 0.062$) for the strong model and +1.0 points (95% CI +0.0 to +3.3; 1 fixed, 0 broken; McNemar $p = 1.000$) for the small one, and the number of wrong answers did not change (2 to 2 and 4 to 4), so it added no false accepts in this sample.

Escalation was broken by a page ban. Letting the cascade escalate when the small model gives up (`cascade_escalate_v3`) escalated 24 questions, yet the strong tier inside the cascade answered only 2 of them correctly while standalone `strong_only_v3` answers 16 of the same 24. The cause is that after a failed verification the graph bans every page that produced a value, so a retry or an escalation cannot use the right page when the failure was a verifier false reject or a

reading slip on it. Keeping failed pages available (`cascade_keep_v3`), the only change, adds +9.0 points (95% CI +3.1 to +16.7; 9 fixed, 0 broken; McNemar $p = 0.004$). Against `strong_only_v3` the repaired cascade differs by -2.0 points (95% CI -8.8 to +4.9; 4 fixed, 6 broken; McNemar $p = 0.754$) at lower cost per question, which is too imprecise to settle the two-point margin of H1. Even so, inside the cascade the strong tier answered 9 of 22 escalated questions against 14 standalone, so a gap remains.

config	n	verified acc. (CI)	abstain	$/question	$/verified correct	p50 s
`small_only_v3`*	100	72.0 (62–81)	32	0.0146	0.0202	19.5
`strong_only_v3`*	100	85.0 (77–92)	21	0.0305	0.0359	13.8
`cascade_escalate_v3`*	100	74.0 (65–83)	27	0.0281	0.0380	16.5
`cascade_keep_v3`*	100	83.0 (74–90)	21	0.0262	0.0315	20.0

Table 3: Exploratory variants (*: designed after reading dev failures).

6.6 Held-out test sample

Following Addendum A of the preregistration (written after the dev results and before any test access), every configuration ran on the same seeded random sample of test questions. Only a preregistered sample of 100 of 295 test questions has been read; the other test questions are unread. The guard log records 10 test reads: one per configuration, plus those of a first attempt that a budget-accounting bug invalidated and that was re-run with unchanged settings. Decision rules were fixed in advance and are applied mechanically below.

config	n	verified acc. (CI)	abstain	$/question	$/verified correct	p50 s
`B0_naive_rag`	100	9.0 (4–15)	92	0.0025	0.0282	2.5
`small_only`	100	58.0 (46–70)	43	0.0131	0.0225	16.8
`strong_only`	100	70.0 (59–81)	29	0.0273	0.0390	13.9
`cascade`	100	58.0 (46–70)	43	0.0160	0.0275	16.3
`small_only_v3`*	100	59.0 (47–71)	42	0.0131	0.0221	16.8
`strong_only_v3`*	100	73.0 (61–84)	26	0.0273	0.0374	17.0
`cascade_keep_v3`*	100	70.0 (57–83)	26	0.0265	0.0379	17.5

Table 4: Held-out test sample (*: exploratory variants, designed on dev).

H3, first half. The small-model agent beats naive RAG by +49.0 points (95% CI +36.6 to +61.1; 50 fixed, 1 broken; McNemar $p < 0.001$).

H1 as preregistered: not supported. `cascade` minus `strong_only` is -12.0 points (95% CI -21.4 to -3.0; 5 fixed, 17 broken; McNemar $p = 0.017$); cost per question $0.0160 against $0.0273. The rule requires the lower end of the interval to be at least -2 points and the cascade to be cheaper.

Exploratory variants. `small_only_v3` minus `small_only`: +1.0 points (95% CI +0.0 to +3.4; 1 fixed, 0 broken; McNemar $p = 1.000$), not confirmed (rule: lower end of the interval above zero); `strong_only_v3` minus `strong_only`: +3.0 points (95% CI +0.0 to +7.0; 3 fixed, 0 broken; McNemar $p = 0.250$), not confirmed (rule: lower end of the interval above zero); `cascade_keep_v3` minus `cascade`: +12.0 points (95% CI +5.6 to +19.4; 12 fixed, 0 broken; McNemar $p < 0.001$),

confirmed (rule: lower end of the interval above zero). 1 of 3 evaluated variants is confirmed; a confirmed variant is promoted to the headline only after the full test run, and the others stay exploratory.

Cost and accuracy of the repaired cascade. `cascade_keep_v3` minus `strong_only_v3` is -3.0 points (95% CI -10.0 to +3.8; 5 fixed, 8 broken; McNemar $p = 0.581$), at $0.0265 against $0.0273 per question.

Dev to test. Verified accuracy, `B0_naive_rag` 10.0% on dev and 9.0% on test; `small_only` 71.0% on dev and 58.0% on test; `strong_only` 80.0% on dev and 70.0% on test; `cascade` 71.0% on dev and 58.0% on test. Splits are chronological, so the test filings are the later ones in the quarter; the shift is reported, not corrected.

6.7 E4: does it know when it does not know?

On the live episodes of `cascade_keep_v3` (exploratory, n=100) the base accuracy is 83.0%; abstaining on the least confident 10% lifts selective accuracy to 92.2%, and at the threshold chosen for a 95% target the agent answers 87.0% of questions with 95.4% accuracy (threshold chosen on the same episodes, so in sample). A logistic calibrator over verifier results, retrieval margin and retries has a cross-validated expected calibration error of 0.084, but it does *not* beat the model's own stated confidence: AURC 0.097 calibrated against 0.047 raw (lower is better). Figure 2 shows both curves.

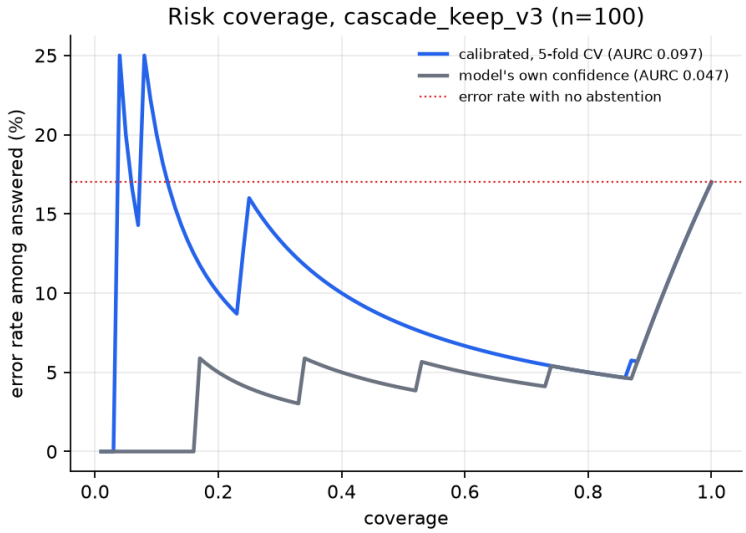


Figure 2: Risk-coverage curves.

6.8 E6: where does it fail?

Attribution was run on the offline rules agent only (117 failures; the live agent has not been attributed because counterfactual replay costs API calls). Failures by stage: intent 33, retrieval 2, rerank 3, read 42, plan or compute 3, citation 1, verifier 33. Retrieval and rerank together explain 4%, so once the reranker is fine-tuned retrieval is no longer the bottleneck. Injecting the gold table values flips 18% of failures, the gold page at rank one 3%, and the gold intent 0%; most

failures are abstentions on answerable questions, which none of these injections addresses. The intent stage is probably a non-causal label (stages are assigned earliest first), and the live agent's stage distribution will differ.

6.9 E8: what does the season show?

Over 245 filings the XBRL arrived a median 72 minutes after the results PDF (quartiles 25 and 164, 95th percentile 317); the PDF came first for 95.1% of filings, and 21.2% of XBRL files arrived more than three hours later. 6.1% of filings had at least one revised XBRL, which the gold handling must follow. This lag is what the self-grading loop uses: answer when the PDF lands, grade when the XBRL arrives.

7 Limitations

- **Small, single-quarter evidence.** All agent results use fixed samples of one hundred questions from a dry run on one quarter, one domain (Ind AS, non bank results) and facts that XBRL covers. Confidence intervals are wide, and several comparisons (for example the repaired cascade against the strong model) are inconclusive at this size.
- **Held-out data.** Only a preregistered sample of 100 of 295 test questions has been read; the other test questions are unread. The guard log records 10 test reads: one per configuration, plus those of a first attempt that a budget-accounting bug invalidated and that was re-run with unchanged settings. The exploratory variants were designed after reading dev failures, so their dev numbers are optimistic; a variant counts as confirmed only under the rule in the preregistration addendum, and a confirmed variant becomes a headline result only after the full test run.
- **Untested hypotheses.** The long-context baseline B1, learning from graded outcomes (H4, H5) and robustness to prompt injection (H6) were preregistered but are not part of this version, so no claim is made for them.
- **Synthetic questions.** Questions are template questions with rule-based surface variants; they test generalisation to new companies and filings, not free-form analyst phrasing. Human validation of the question sample is pending.
- **Offline results.** Rows labelled offline come from a deterministic backend, not a language model; failure attribution was run only on that backend.
- **Data quality.** Many PDFs are scans with an embedded OCR layer, and filings without a detected results page contribute no questions. One exchange PDF was truncated at the source and excluded. Gold values are XBRL except where adjudication overrides them.
- **Cost accounting.** Costs are real API spend recorded at the price of the original call; the pre-run estimator was conservative by a wide margin and the ledger is authoritative.

8 Conclusion

A free, automatic answer key that arrives after the document turns a static QA task into a loop: answer, verify, get graded. In this first evaluation the largest effects are the ones a verifiable design makes visible: a fine-tuned reranker on free labels removes retrieval as the bottleneck, the agent is far more accurate than naive RAG, and the remaining loss sits in the verifier and the retry

logic rather than in the models. The specified cascade did not beat a single small model and is significantly worse than the strong model on the held-out sample, so H1 is not supported. Paired experiments on dev located two defects; the fix that keeps failed pages available is confirmed on held-out questions (+12 points over the specified cascade), and the repaired cascade matches the strong model there, but at only about 3% lower cost per question, against 14% on dev. All agent configurations lose 10 to 13 points from dev to test. The next steps are the full test split, the long-context baseline, learning from graded outcomes, and a prompt-injection evaluation.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[3] Gordon V Cormack, Charles L A Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of SIGIR*, 2009.

[4] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.

[5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *Proceedings of ICML*, 2017.

[7] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720*, 2024.

[8] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. Financebench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.

[9] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

[10] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019.

[11] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.